

거리두기 - 풀이

작성자: 장근영

1번 학생의 위치 B_1 을 x 로 정했다고 하자.

번호가 연속한 두 학생은 K 이상의 거리를 두어야 하므로, 모든 i 에 대해

$$B_i \geq B_{i-1} + K \geq B_{i-2} + 2K \geq \dots \geq B_1 + (i-1)K = x + (i-1)K$$

가 성립한다. 따라서 $B_1 = x$ 인 수열 중 각 학생을 가장 왼쪽에 배치한 수열은 $B_i = x + (i-1)K$ 이다.

어떤 위치 i 에서 $x + (i-1)K > A_i$ 라면, $B_1 = x$ 인 어떤 수열도 조건을 만족할 수 없다. 반대로 모든 i 에 대해 $x + (i-1)K \leq A_i$ 라면, 수열 $B_i = x + (i-1)K$ 자체가 모든 조건을 만족한다.

즉, 시작 위치 x 가 가능한지 여부는 모든 i 에 대해 $x + (i-1)K \leq A_i$ 인지 확인하여 판별할 수 있다.

부분문제 1

모든 $1 \leq i < N$ 에 대해 $A_{i+1} - A_i \geq K$ 가 성립한다.

따라서 수열 A 자체가 문제의 조건을 만족한다. 또한 어떤 수열 B 를 선택하더라도 $B_1 \leq A_1$ 이어야 한다.

수열 A 를 그대로 선택하면 $B_1 = A_1$ 을 달성할 수 있으므로, 이는 가능한 가장 큰 시작 위치이다. 따라서 모든 i 에 대해 $B_i = A_i$ 로 출력하면 된다.

전체 시간 복잡도는 $\mathcal{O}(N)$ 이다.

부분문제 2

$K = 0$ 이다.

앞에서 설명한 것처럼 시작 위치를 B_1 로 정했을 때, 각 학생을 가장 왼쪽에 배치한 수열은 $B_i = B_1 + (i-1)K = B_1$ 이다.

모든 i 에 대해 $B_1 = B_i \leq A_i$ 여야 하므로 $B_1 \leq \min_{1 \leq i \leq N} A_i$ 이 성립해야 한다.

따라서 가능한 가장 큰 시작 위치는 $B_1 = \min_{1 \leq i \leq N} A_i$ 이며, 모든 i 에 대해 $B_i = B_1$ 로 출력하면 된다.

전체 시간 복잡도는 $\mathcal{O}(N)$ 이다.

부분문제 3

조건을 만족하면서 $0 \leq B_1 \leq 100$ 인 수열이 하나 이상 존재한다.

또한 항상 $B_1 \leq A_1 \leq 100$ 이므로, 최적의 시작 위치도 0 이상 100 이하이다.

따라서 $x = 0, 1, \dots, 100$ 을 각각 시작 위치의 후보로 두고, 모든 i 에 대해 $x + (i-1)K \leq A_i$ 인지 확인한다.

조건을 만족하는 후보 중 가장 큰 값을 x^* 라고 하자. 그러면 모든 i 에 대해

$$B_i = x^* + (i - 1)K$$

로 출력하면 된다.

후보가 101개이고 각 후보를 검사하는 데 $\mathcal{O}(N)$ 의 시간이 필요하므로, 전체 시간 복잡도는 $\mathcal{O}(N)$ 이다.

부분문제 4

전체 문제에서는 최적의 B_1 이 음수일 수 있으므로 음수인 시작 위치도 확인해야 한다.

먼저 확인해야 할 시작 위치의 범위를 구하자.

항상 $B_1 \leq A_1 \leq \max_i A_i$ 이므로, $\max_i A_i$ 보다 큰 값은 확인할 필요가 없다.

한편 $x = -NK$ 라고 하면, 모든 $1 \leq i \leq N$ 에 대해

$$x + (i - 1)K = -(N - i + 1)K \leq 0 < A_i$$

이다. 따라서 $x = -NK$ 는 반드시 가능한 시작 위치이다.

그러므로 최적의 시작 위치는 $-NK$ 이상 $\max_i A_i$ 이하이다.

이 범위의 모든 정수 x 를 차례로 확인한다. 각 후보에 대해 모든 i 에서 $x + (i - 1)K \leq A_i$ 가 성립하는지 검사하고, 조건을 만족하는 가장 큰 값을 x^* 로 선택한다.

이후 모든 i 에 대해 $B_i = x^* + (i - 1)K$ 로 출력한다.

확인하는 시작 위치는 $-NK$ 부터 $\max_i A_i$ 까지이므로 그 개수는 $\mathcal{O}(NK + \max_i A_i)$ 이다. 각 후보를 검사하는 데 $\mathcal{O}(N)$ 의 시간이 필요하다.

따라서 전체 시간 복잡도는

$$\mathcal{O}\left(N(NK + \max_i A_i)\right)$$

이고, 공간 복잡도는 $\mathcal{O}(N)$ 이다.

문제의 제한에서는 시작 위치 후보가 최대 약 1100개이므로 충분히 빠르다.

별해: 뒤에서부터 수열 구성하기

후보를 하나씩 확인하지 않고, 수열 B 를 뒤에서부터 직접 정하면 선형 시간에 해결할 수 있다.

먼저 N 번 학생의 위치는 $B_N \leq A_N$ 만 만족하면 된다. B_N 이 클수록 앞의 학생들을 더 오른쪽에 세울 수 있으므로, $B_N = A_N$ 으로 정하는 것이 최적이다.

이제 $i = N - 1, N - 2, \dots, 1$ 의 순서로 B_i 를 정하자.

i 번 학생은 위치 A_i 보다 오른쪽에 설 수 없으므로 $B_i \leq A_i$ 여야 한다. 또한 $B_{i+1} - B_i \geq K$ 여야 하므로 $B_i \leq B_{i+1} - K$ 여야 한다.

따라서 가능한 가장 큰 B_i 는 $B_i = \min(A_i, B_{i+1} - K)$ 이다.

이와 같이 각 B_i 를 가능한 한 크게 정하는 것이 항상 유리하다. B_i 가 클수록 그 앞의 학생이 만족해야 하는 조건 $B_{i-1} \leq B_i - K$ 의 오른쪽 값도 커지므로, 앞의 학생을 배치하는 데 불리해지지 않기 때문이다.

따라서 다음과 같이 수열을 구성한다.

- $B_N = A_N$
- $i = N - 1, N - 2, \dots, 1$ 에 대해 $B_i = \min(A_i, B_{i+1} - K)$

구성한 수열은 정의에 의해 모든 i 에 대해 $B_i \leq A_i$ 를 만족한다. 또한 $B_i \leq B_{i+1} - K$ 이므로 $B_{i+1} - B_i \geq K$ 도 만족한다.

각 단계에서 현재 위치를 가능한 한 크게 정했으며, 더 큰 값을 선택하면 $B_i \leq A_i$ 또는 $B_i \leq B_{i+1} - K$ 중 하나를 위반한다. 따라서 마지막으로 얻은 B_1 은 가능한 최댓값이다.

전체 시간 복잡도는 $\mathcal{O}(N)$ 이고, 수열을 저장하는 공간 복잡도는 $\mathcal{O}(N)$ 이다.



주사위 탑 쌓기 - 풀이

작성자: 박상훈

부분문제 1

두 주사위를 하나의 탑으로 쌓을 수 있으려면, 아래쪽 주사위의 윗면에 적힌 수와 위쪽 주사위의 아랫면에 적힌 수가 같아야 한다.

윗면에 A_i 가 적힌 주사위의 아랫면에는 $7 - A_i$ 가 적혀 있으므로, 두 주사위를 하나의 탑으로 쌓을 수 있는 조건은

$$A_1 + A_2 = 7$$

이다. 이 조건을 만족하면 1, 그렇지 않으면 2를 출력한다.

부분문제 2

윗면에 3이 적힌 주사위의 개수를 C_3 , 윗면에 4가 적힌 주사위의 개수를 C_4 라고 하자.

윗면에 3이 적힌 주사위와 윗면에 4가 적힌 주사위는 번갈아 쌓을 수 있다. 두 개수가 같다면 모든 주사위를 하나의 탑으로 쌓을 수 있다.

두 개수가 다르다면, 더 많이 있는 종류의 주사위가 $|C_3 - C_4|$ 개 더 많다. 하나의 탑에서 한 종류의 주사위는 다른 종류보다 최대 1개 더 많이 사용될 수 있으므로, 필요한 탑의 최소 개수는 $|C_3 - C_4|$ 개이다.

따라서 정답은

$$\max(1, |C_3 - C_4|)$$

이다.

부분문제 3

각 x 에 대해, 윗면에 x 가 적힌 주사위의 개수를 C_x 라고 하자.

같은 탑에 들어갈 수 있는 주사위의 윗면에 적힌 수는 각각 다음 세 쌍 중 하나에 속한다.

$$(1, 6), \quad (2, 5), \quad (3, 4)$$

서로 다른 쌍에 속하는 주사위는 같은 탑에 쌓을 수 없으므로, 각 쌍을 따로 생각할 수 있다.

부분문제의 조건에 따라 C_x 와 C_{7-x} 는 항상 서로 다르다. 부분문제 2와 같은 방법을 각 쌍에 적용하면 정답은

$$|C_1 - C_6| + |C_2 - C_5| + |C_3 - C_4|$$

이다.

부분문제 4

부분문제 3과 마찬가지로 $(1, 6)$, $(2, 5)$, $(3, 4)$ 를 각각 따로 생각한다.

한 쌍에 속한 두 종류의 주사위 개수를 P , Q 라고 하자. 두 개수가 모두 0이면 이 쌍에서는 답을 만들 필요가 없다. 그렇지 않고 두 개수가 같다면, 모든 주사위를 번갈아 쌓아 하나의 탑을 만들 수 있다. 두 개수가 다르다면 필요한 탑의 최소 개수는 $|P - Q|$ 이다.

따라서 각 쌍이 정답에 더하는 값은

$$\begin{cases} 0 & (P = Q = 0), \\ \max(1, |P - Q|) & (\text{그 외}) \end{cases}$$

이다. 세 쌍에 대해 이 값을 모두 더하면 정답을 구할 수 있다.

시간복잡도는 $O(N)$ 이다.

간식 분배 - 풀이

작성자: 박영우

부분문제 1

모든 학생은 정확히 하나의 간식을 좋아하므로, 학생이 방에 들어갔을 때 최대 하나의 간식을 가져간다.

만약 여러 학생이 공통으로 좋아하는 간식이 있다면, 그중 나중에 방에 들어가는 학생은 아무 간식도 가져갈 수 없다. 따라서 이 경우에는 목표를 달성할 수 없다.

그렇지 않다면 모든 학생이 서로 다른 간식을 좋아하므로, 학생들을 어떤 순서로 방에 들여보내더라도 모든 학생이 정확히 하나의 간식을 가져간다.

부분문제 2

이 부분문제에서는 목표를 달성하는 것이 가능할 경우, $1, 2, \dots, N$ 번 학생들을 이 순서대로 방에 들여보내도 목표를 달성할 수 있다.

따라서 방 안에 남은 간식을 관리하면서, 각 학생이 순서대로 방에 들어갈 때 가져가는 간식의 개수를 세면 된다.

부분문제 3

이 부분문제에서 학생들이 방에 들어가는 경우의 수는 최대 $N! \leq 120$ 가지이다. 따라서 각각의 경우에 대해 부분문제 2의 알고리즘을 사용하여 모든 학생이 정확히 하나의 간식을 가져가는지 판정하면 된다.

부분문제 4

$\{1, 2, \dots, N\}$ 의 모든 부분집합 S 에 대해, $dp[S]$ 를 다음과 같이 정의한다.

- 만약 S 에 속한 학생들의 순서를 잘 정하여, 그 순서대로 방에 들여보냈을 때 S 에 속한 각 학생이 정확히 하나의 간식을 가져가도록 할 수 있다면 $dp[S] = 1$ 이다.
- 그렇지 않다면 $dp[S] = 0$ 이다.

모든 S 에 대하여, $dp[S] = 1$ 인 조건은 다음과 같다.

- S 가 비어 있거나,
- S 에서 i 를 제거한 집합 T 가 있어서 $dp[T] = 1$ 이고, i 번 학생이 좋아하는 간식 중 T 에 속한 학생들이 좋아하는 간식이 아닌 것은 정확히 1개이다.

따라서, S 의 모든 부분집합 T 에 대해 $dp[T]$ 를 구했다면 $dp[S]$ 는 $\mathcal{O}(N)$ 에 구할 수 있다.

비트필드를 이용한 다이내믹 프로그래밍을 사용하면 $\mathcal{O}(N2^N)$ 에 풀 수 있다.

부분문제 5

현재 일부 학생들이 이미 방에 들어가 정확히 하나의 간식을 가져갔고, 남아 있는 간식 중 정확히 하나만 좋아하는 학생 i 가 있다고 하자.

현재 상태에서 남은 학생들의 순서를 정하여 모든 학생들이 정확히 하나의 간식을 가져가도록 할 수 있다고 하자. 그 순서에서 i 보다 먼저 들어오는 학생은 i 가 가져갈 유일한 간식을 좋아할 수 없다. 따라서 i 를 그 학생들보다 앞으로 옮겨도 다른 학생들이 가져가는 간식에 영향을 주지 않는다. 그러므로, 만약 현재 상태에서 목표를 달성하는 것이 가능하다면 i 를 바로 들여보내도 답이 존재한다.

따라서 다음과 같은 방법으로 문제를 풀 수 있다.

- 현재 남은 간식 중 정확히 하나를 좋아하는 학생이 없다면 불가능하다고 판정한다.
- 그렇지 않은 경우, 정확히 하나의 간식을 좋아하는 학생을 찾고, 그 학생을 방에 들여보낸다. 만약 그러한 학생이 여러 명일 경우, 어떤 학생을 먼저 들여보내더라도 상관없다.
- 위 과정을 모든 학생이 간식을 가져갈 때까지 반복한다.

부분문제 5의 경우, $N \leq 300$ 이다. 다음과 같이 구현할 수 있다.

- 학생과 간식의 관계를 $N \times N$ 배열로 저장한다. i 번 학생이 j 번 간식을 좋아한다면 i 행 j 열에 1을 저장한다.
- 아직 방에 들어가지 않은 모든 학생의 행을 확인하여, 현재 남은 선호 간식이 정확히 하나인 학생을 찾는다. 그러한 행이 존재하지 않는다면 불가능하다고 판정한다.
- 찾은 행을 i 번 행이라 하고, i 번 학생이 좋아하는 간식 중 현재 유일하게 남아있는 것을 j 번 간식이라 하자. 배열에서 j 열을 모두 지운다.
- 위 과정을 모든 학생이 방에 들어갈 때까지 총 N 번 반복한다.

한 단계에 $\mathcal{O}(N^2)$ 번의 연산이 사용되고, 총 N 번 반복해야 하므로 전체 시간 복잡도는 $\mathcal{O}(N^3)$ 이다.

부분문제 6

부분문제 5의 알고리즘을 최적화할 수 있다. 각 학생 i 에 대해 i 번 학생이 좋아하는 간식 중 현재 방에 남은 것의 개수를 cnt_i 라 하고, 이 배열을 관리한다. 다음과 같이 해결할 수 있다.

- 방에 들어갈 학생을 구하는 것은 아직 방에 들어가지 않은 학생 중 $cnt_i = 1$ 인 학생 i 를 구하면 된다. 이때 i 번 학생이 방에 들어가면 정확히 하나의 간식 j 를 가져가게 된다.
- 이 과정에서 j 번 간식을 좋아하는 학생 k 에 대해, cnt_k 가 1 감소한다. j 번 간식을 좋아하는 모든 학생 k 를 찾아 cnt_k 를 갱신한다.

두 과정 모두 $\mathcal{O}(N)$ 에 처리할 수 있고, 총 N 번 반복하므로 전체 시간 복잡도는 $\mathcal{O}(N^2)$ 이다.

부분문제 7

부분문제 6의 알고리즘에서 추가적인 최적화를 할 수 있다.

- $cnt_i \leq 1$ 인 학생들을 큐(queue) 등의 자료구조를 사용해 저장한다. 어떤 학생이 간식을 가져가서 학생 i 의 cnt_i 가 1 감소했다 하자. 이때 $cnt_i \leq 1$ 인지 검사하고, 만약 그렇다면 i 를 큐에 추가하면 된다.
- j 번 간식을 좋아하는 학생들을 구하는 것은, 처음에 간식 j ($1 \leq j \leq N$) 각각에 대해 j 번 간식을 좋아하는 학생들의 배열을 `std::vector` 등의 동적 배열을 만들어 사용하면 된다. 각 간식은 최대 한 번 제거되므로, 각 배열도 최대 한 번 확인된다.

전체 시간복잡도는 $\mathcal{O}(N + \sum C_i)$ 이다.

게임 - 풀이

작성자: 박영우

각 j ($1 \leq j \leq Q$)에 대하여, j 번째 게임에서 s_j 번 방에 출구가 있는 경우 앨리스가 승리한다. 이러한 경우를 먼저 처리한다. 앞으로의 서술에서는 s_j 번 방에 출구가 없다는 것을 가정한다.

부분문제 1

각 j ($1 \leq j \leq Q$)에 대하여, j 번째 게임에서 $s_j = 1$ 일 경우 앨리스가 승리한다. 그렇지 않을 경우 앨리스가 승리할 조건은 다음과 같다.

- $c_1, c_2, \dots, c_{s_j-1}$ 의 최솟값이 k_j 보다 크거나 같다.

위 조건이 성립하면, 앨리스는 1번 방에 도착하기 전까지 현재 방에서 번호가 정확히 1 줄어드는 방으로 향하는 통로를 k_j 개 선택할 수 있다. 따라서 앨리스가 승리한다.

그렇지 않을 경우, $c_t < k_j$ 이며 $1 \leq t < s_j$ 인 정수 t 가 존재한다. 이 경우 밥은 앨리스가 $t+1$ 번 방에 도달하기 전까지는 아무 통로를 선택하다가, $t+1$ 번 방에 도달했을 때 $t+2$ 번 방으로 향하는 통로를 선택하면 된다. (만약 $t+1 = N$ 이라면, 앨리스는 다른 방으로 이동할 수 없으므로 승리할 수 없다) 그러면 앨리스는 t 번 방으로 이동할 수 없고, s_j 번 방에서 시작하여 1번 방에 도달하기 위해서는 반드시 t 번 방을 거쳐야 하므로 앨리스는 승리할 수 없다.

$i = 1, 2, \dots, N-1$ 에 대해 $\min(c_1, c_2, \dots, c_i)$ 를 $\mathcal{O}(N)$ 에 전처리해 두면 각각의 게임에 대해 앨리스가 승리하는지 $\mathcal{O}(1)$ 에 판정할 수 있다.

부분문제 2

이 부분문제에서 1번 방은 다른 모든 방과 통로로 직접 연결되어 있고, 1번 방이 아닌 다른 두 방을 연결하는 통로는 없다.

1번 방에 출구가 있는 경우와 없는 경우로 나눠서 해결한다.

1번 방에 출구가 있는 경우, 각 게임에서 앨리스가 승리할 수 있다면 1번 방에 도달할 수 있다. 따라서 각 게임에 대해 1번 방에 도달할 수 있는지만 검사해도 충분하다.

1번 방에 출구가 없었다 하자. j 번째 게임에 대해 앨리스가 출구가 있는 방에 도달할 수 있다면, 우선 1번 방에 도달할 수 있어야 한다. 우선 이 조건을 검사하고, 성립하지 않는다면 앨리스가 패배한다고 판정한다.

1번 정점에 앨리스가 있고, 이 시점에서 앨리스가 승리할 수 있다면 정확히 한 번의 이동을 통해 출구가 있는 방에 도달할 수 있어야 한다. 그렇지 않다면, 앨리스는 1번 방이 아닌 다른 방으로 오게 되고, 앨리스는 다시 1번 방으로 돌아와야 한다. (혹은 해당 방에 연결된 통로의 수가 적어 더 이상 이동할 수 없다) 밥은 통로를 잘 선택하여 앨리스가 1번 방과 출구가 없는 다른 방을 무한히 왕복하도록 하게 할 수 있고, 따라서 앨리스는 승리할 수 없다.

따라서 1번 방에서 정확히 한 번의 이동을 통해 출구가 있는 방으로 이동할 수 있는지 판정하면 된다. 앨리스가 출구가 없는 방으로 향하는 통로를 고른다면, 밥은 그 통로를 선택하여 앨리스가 출구가 없는 방으로 이동하게 할 수 있다. 따라서 앨리스는 1번 방에 있을 때 출구가 있는 방으로 향하는 통로만을 골라야 한다. j 번째 게임에서 앨리스가 1번 방에 왔을 때, k_j 가 그러한 통로의 수보다 많다면 승리할 수 없고, 그렇지 않다면 승리할 수 있다.

부분문제 3

각 게임에 대해 $k_j = 1$ 이므로, 앨리스는 각 게임에서 정확히 하나의 통로를 고른다. 따라서 밥의 선택과 무관하게 앨리스는 매 이동에서 원하는 인접한 방으로 이동할 수 있다.

따라서, 앨리스가 j 번 게임에서 이길 필요충분조건은 s_j 번 방과 출구가 있는 방이 통로를 통해 연결되어 있는 것이다.

방과 통로들을 그래프로 해석하면, 출구가 있는 방과 s_j 번 방이 같은 연결 요소(connected component)에 있는 것이다.

처음에 주어진 방들을 dfs나 Union-Find 등의 알고리즘을 사용해 같은 연결 요소에 속한 방끼리 분류하고, 각 연결 요소에 출구가 있는지 구하면 쿼리를 $\mathcal{O}(1)$ 의 시간복잡도에 처리할 수 있다.

전체 시간복잡도는 $\mathcal{O}(N + M + Q)$, Union-Find를 사용할 경우 $\mathcal{O}((N + M + Q)\alpha(N))$ 이다.

부분문제 4

양의 정수 k 에 대해 V_k 를 다음과 같이 정의한다.

- 앨리스가 게임에서 k 개의 통로를 선택할 수 있다고 할 때, 앨리스가 게임에서 승리하게 되는 시작 방 번호의 집합은 V_k 이다.

즉, 각 j 에 대해 $s_j \in V_k$ 라면 앨리스가 승리하고, 그렇지 않으면 패배한다.

양의 정수 k 에 대해, 다음과 같은 관찰을 할 수 있다.

- v 번 방에 출구가 있다면 $v \in V_k$ 이다.
- v 와 연결된 통로 중, V_k 에 있는 방으로 향하는 통로가 k 개 이상이라 하자. 그렇다면 $v \in V_k$ 이다.

따라서, V_k 를 구하기 위해 다음과 같은 알고리즘을 생각할 수 있다.

1. 집합 S 를 공집합으로 초기화한다. 각 방 v 에 대해, d_v 를 v 와 인접한 통로 중 S 에 있는 방으로 향하는 통로의 개수로 정의한다. 처음에 d_v 는 모두 0이다.
2. 출구가 있는 방 v 에 대하여 v 를 S 에 추가한다. 이때 v 와 c 개의 통로로 인접한 방 u 에 대해 d_u 에 c 가 더해진다. v 와 인접한 모든 방 u 에 대해 d_u 를 갱신한다.
3. 만약 어떤 방 v 에 대해 v 가 S 에 없고, $d_v \geq k$ 라면 v 를 S 에 추가한다. 전과 같은 규칙으로 v 와 인접한 방 u 들에 대해 d_u 를 갱신한다.
4. 이 과정을 S 에 없는 모든 방 v 에 대해 $d_v < k$ 가 될 때까지 반복한다.

두 번째 과정이 끝난 이후, S 는 출구가 있는 방의 집합이다. 따라서 이 시점에서 $S \subseteq V_k$ 이다.

세 번째 과정에서 S 에 추가된 모든 v 에 대해 $d_v \geq k$ 이다. 즉, v 와 연결된 통로 중 v 가 추가되기 직전 시점의 S 에 속하는 방과 연결된 것이 k 개 이상이다. 따라서 v 를 추가하기 직전 $S \subseteq V_k$ 였다면, v 를 추가한 이후에도 $S \subseteq V_k$ 이다.

따라서 위 알고리즘이 종료되었을 때, $S \subseteq V_k$ 이다.

이제 $V_k \subseteq S$ 를 증명한다. S 에 없는 방 v 가 V_k 에도 없음을 보이면 충분한데, 이는 S 에 없는 방 v 에 대하여 앨리스가 v 에서 시작했을 때 승리할 수 없음을 의미한다. 이는 다음과 같이 증명할 수 있다.

- v 번 방과 연결된 통로가 k 개 미만일 경우, 앨리스는 다른 방으로 이동할 수 없는데 v 번 방에는 출구가 없으므로($v \notin S$) 앨리스가 승리할 수 없다.
- 그렇지 않을 경우, v 번 방과 연결된 통로 중 S 에 속하는 방으로 향하는 것은 k 개 미만이다. 따라서, 앨리스가 어떻게 통로를 선택하더라도, 그 통로 중 적어도 하나는 S 에 없는 방으로 향한다.
- 밥이 그러한 통로를 반복해서 선택하는 전략을 사용할 경우, 앨리스가 어떻게 이동하더라도 S 에 속한 방으로 이동할 수 없다. 출구가 있는 모든 방은 S 에 속하므로, 앨리스는 출구가 있는 방에 도달할 수 없어 승리할 수 없다.

따라서, $V_k = S$ 이다.

이 부분 문제에서는 모든 게임에 대해 k_j 가 동일하므로, 한 V_k 만 구하면 충분하다. 앞서 제시한 알고리즘을 그대로 사용할 수 있는데, d_u 가 k 이상이며 S 에 속하지 않는 방 u 들을 큐(queue) 등의 자료구조를 사용하여 관리하면 충분하다.

위 알고리즘에서 방 u 가 S 에 추가되었을 때, u 와 연결된 통로들이 한 번 순회된다. 따라서 각 $i(1 \leq i \leq M)$ 에 대해, 방 a_i 와 b_i 를 연결하는 통로들은 최대 두 번 확인된다.

전체 시간복잡도는 $\mathcal{O}(N + M + Q)$ 이다.

부분문제 5

이 부분문제에서는 모든 게임에 대해 앨리스가 시작하는 방 s_j 가 같다. $s_1 = s_2 = \dots = s_Q = s$ 라 하자.

다음과 같은 관찰을 할 수 있다.

- 모든 양의 정수 i 에 대하여, $V_{i+1} \subseteq V_i$ 이다.

부분문제 4의 알고리즘에서 S 에 정점이 추가되는 조건은 k 가 증가함에 따라 더 강해지는데, 따라서 $k = i + 1$ 일 때 알고리즘에서 v 가 S 에 추가되었다면 $k = i$ 일 때에도 추가된다.

따라서, $s \in V_k$ 인 최대 k 가 존재한다면, $1 \leq k' \leq k$ 인 모든 k' 에 대해 $s \in V_{k'}$ 이다.

부분문제 4의 알고리즘을 사용하여 한 k 에 대해 V_k 를 구할 수 있으므로, 이분 탐색으로 $s \in V_k$ 인 k 를 찾거나, 존재하지 않는지 판정할 수 있다.

이후 각 게임 j 에 대해서 k_j 와 k 의 대소를 비교하여 앨리스가 승리하는지 판정할 수 있다.

모든 게임에서 주어지는 k_j 중 최댓값을 X 라 하자. $k = 1, 2, \dots, X$ 범위에 대해서만 이분 탐색을 적용해보면 충분하므로, 전체 시간복잡도는 $\mathcal{O}((N + M) \log X + Q)$ 이다.

부분문제 6

이 부분문제에서 $N, M, k_j \leq 3000$ 이다. 따라서 $X \leq 3000$ 이다.

양의 정수 $k = 1, 2, \dots, X$ 에 대해 V_k 를 각각 구하면, 모든 게임에 대하여 앨리스가 승리하는지 $\mathcal{O}(1)$ 에 판정할 수 있다.

전체 시간복잡도는 $\mathcal{O}((N + M)X + Q)$ 이다.

부분문제 7

출구가 없는 각 방 v 에 대해, $v \in V_k$ 를 만족하는 최대 k 를 K_v 라 하자. 그러한 k 가 없다면 $K_v = 0$ 으로 둔다. 또한, v 에 출구가 있다면 $K_v = 10^{18}$ 로 둔다. 그러면 각 게임 j 에서 엘리스가 승리할 필요충분조건은 $k_j \leq K_{s_j}$ 이다.

모든 K_v 를 한 번에 구하기 위해 부분문제 4의 알고리즘을 k 가 큰 값에서 작은 값으로 감소시키며 실행한다.

현재까지 V_k 에 포함되었다고 판정한 방들의 집합을 S 라 하자. S 에 포함되지 않은 각 방 v 에 대해, d_v 를 v 와 S 에 속한 방들을 연결하는 통로의 수로 정의한다.

처음에는 출구가 있는 방들을 모두 S 에 넣고, 현재 k 를 10^{18} 로 둔다.

현재 k 에 대해 $d_v \geq k$ 인 방 v 가 있다면, 부분문제 4의 관찰에 따라 v 도 V_k 에 포함된다. 따라서 이러한 방을 S 에 추가하고, 인접한 방들의 d 값을 갱신한다. 새롭게 조건을 만족하는 방이 생길 수 있으므로, 그러한 방들을 큐를 사용하여 관리하고, 더 이상 생기지 않을 때까지 반복한다.

더 이상 S 에 추가할 방이 없다고 하자. 이때 S 에 속하지 않은 방들의 d_v 중 최댓값을 x 라 하면, $x < k' \leq k$ 인 모든 k' 에 대해 $V_{k'} = S$ 이다. 또한 $d_v = x$ 인 방들은 $k = x$ 일 때 처음으로 V_k 에 포함될 수 있다.

따라서 현재 S 에 속하지 않은 방 중 d_v 가 가장 큰 방을 찾아, 현재 k 를 그 값으로 낮추고 다시 같은 과정을 반복하면 된다. 이때 새롭게 S 에 추가되는 방 v 에 대해 $K_v = k$ 로 둔다.

S 에 속하지 않은 방 중 d_v 가 가장 큰 방을 빠르게 찾는 것은 우선순위 큐 등을 사용하여 처리할 수 있다. d_v 가 갱신될 때마다 (d_v, v) 를 우선순위 큐에 삽입하고, d_v 가 가장 큰 방을 찾을 때에는 우선순위 큐에서 가장 큰 값을 꺼내면 된다. 이때 이미 S 에 포함된 방에 대한 원소는 무시하고, v 가 S 에 포함되지 않는 (d_v, v) 가 나올 때까지 반복하면 된다.

$K_v = 0$ 인 방의 경우, d_v 가 한 번도 갱신되지 않으므로 위 과정에서 처리되지 않는다. 이러한 방들에 대해서는 따로 처리한다.

각 방은 S 에 한 번만 추가되고, 각 통로 정보는 양 끝 방이 추가될 때에만 필요하므로 최대 두 번 확인된다. 이로 인해 우선순위 큐에는 $\mathcal{O}(M)$ 개의 원소가 삽입된다.

따라서 전체 시간복잡도는 $\mathcal{O}((N + M) \log(N + M) + Q)$ 이다.